# Material Stream Identification System

TECHNICAL REPORT & PIPELINE OVERVIEW

**Executive Summary:** The goal of this system is to classify waste material images into 7 distinct classes using handcrafted features and classical machine learning models (SVM and k-NN). The pipeline is structured for reproducible training and real-time webcam inference. Currently, the Support Vector Machine (SVM) achieves an overall accuracy of 0.8242 (0.7769 on primary classes).

## Motivation & Problem Statement

Waste sorting is a practical computer vision problem where inputs are often small, noisy, and visually similar across classes. Rather than relying on end-to-end deep learning, this project implements handcrafted image descriptors alongside classical ML to build a compact, explainable classifier for material recognition.

The core requirements addressed by this pipeline include:

- Building a multi-class material classifier using classical machine learning methodologies.
- Preserving a clean train/test workflow, applying augmentations strictly to the training split.
- Deploying a real-time camera application that predicts material classes from a live frame.

## Dataset & Classification Categories

The system classifies items into six primary categories, plus an additional `unknown` class intended to represent synthetic or hard-to-classify samples. This allows the deployment stage to safely reject low-confidence predictions.

| Class | Description |
| --- | --- |
| **Cardboard** | Packaging and box-like material |
| **Glass** | Bottles, jars, and similar transparent or reflective waste |
| **Metal** | Cans and metallic objects |
| **Paper** | Paper sheets, printed pages, and paper waste |
| **Plastic** | Bottles, wrappers, containers, and synthetic plastic waste |
| **Trash** | Mixed waste that does not fit the recyclable categories |
| **Unknown** | Safety fallback for ambiguous/low-confidence samples |

## System Architecture

The workflow is implemented sequentially, producing persisted artifacts at each stage to ensure absolute reproducibility rather than relying on one-off notebook experiments.

```
Raw Images
    |
    ▼
Train/Test Split
    |
    ├───────────────▶ Data Augmentation (train only)
    |
    ├───────────────▶ Unknown Class Generation
    |
    ▼
Feature Extraction
    |
    ▼
Feature Scaling + Label Encoding
    |
    ├───────────────▶ SVM Training
    |
    ├───────────────▶ k-NN Training
    |
    ▼
Evaluation + Reports
    |
    ▼
Real-Time Webcam Inference
```

## Feature Engineering

The project leverages a handcrafted feature vector built from multiple image descriptors, keeping the system lightweight, interpretable, and compatible with classical ML models. Waste categories often differ by material texture, shape, and color distribution rather than fine-grained semantic content. The extracted features include:

- **HOG** for shape and gradient structure
- **Color histograms** mapped across BGR, HSV, and LAB spaces
- **HSV color moments**
- **Local Binary Pattern (LBP)** and **GLCM** statistics for texture mapping
- **Hu moments** for invariant shape detection
- **Canny edge density** and contour statistics
- **Gabor filter responses** for texture analysis

## Model Training & Evaluation

Two classical models are trained using the scaled feature vectors and identical encoded labels. Both pipelines utilize saved `scaler` and `label_encoder` artifacts.

- **Support Vector Machine (SVM):** Acts as the primary classifier utilizing strong margin-based separation. Tuned via `GridSearchCV` over `C`, `gamma`, and `kernel`.
- **k-Nearest Neighbors (k-NN):** Utilized as a comparative baseline model. Tuned via `GridSearchCV` over `n_neighbors`, distance metric, and weighting.

### Current Performance Results

| Model | All-class Accuracy | All-class Macro F1 | Known-class Accuracy |
|---|---|---|---|
| **SVM** | 0.8242 | 0.7972 | 0.7769 |
| **k-NN** | 0.7542 | 0.7024 | 0.6882 |

> **Important Limitation:** While the project is structurally complete and fully reproducible, the current saved models do not yet reach the target assignment threshold of **0.85 validation accuracy** on the primary six classes. Performance is currently bottlenecked by feature quality and the inherent capacity limits of classical models compared to deep learning architectures.

## Real-Time Deployment

The system is deployed via a local webcam application (`src/realtime_app.py`). It utilizes OpenCV for capture, crops a central region of interest, and scales the frame using the pre-fit scaler. Based on a defined confidence threshold, it either assigns one of the six primary classes or falls back to `unknown`. The GUI presents the predicted label, confidence score, top alternatives, and current FPS.

## Future Work & Recommendations

- **Dimensionality Reduction:** Apply advanced feature selection or PCA to streamline the handcrafted feature space and reduce noise.
- **Alternative Architectures:** Evaluate a tuned linear or calibrated classifier on the current features, or introduce a lightweight CNN (e.g., MobileNet) as a baseline comparison.
- **Validation Strategy:** Implement a proper hold-out validation split specifically for model selection, separate from the final test reporting pipeline.